

GraphRAG

May 11, 2025

1 Bài toán:

Xây dựng knowledge graph và đánh giá chất lượng câu trả lời dựa của GraphRAG và NaiveRAG trên tập dữ liệu multi-hop QA

Mục Tiêu:

- Cài đặt và thử nghiệm GraphRAG và NaiveRAG trên tập dữ liệu multi-hop QA sử dụng thư viện lightrag
- Trực quan hóa Knowledge Graph
- Thử nghiệm query trên graph lấy ra những nút (thực thể) và cạnh (mối quan hệ) liên quan tới câu queries.
- Đánh giá chất lượng câu trả lời GraphRAG và NaiveRAG trên tập dữ liệu bằng LLM trên các tiêu chí có sẵn.

Problems:

- Multihop RAG (Retrieval-Augmented Generation đa bước) giải quyết một thách thức then chốt trong hệ thống truy xuất thông tin: trong thực tế, phần lớn câu trả lời không nằm gọn trong một đoạn văn đơn lẻ mà cần kết hợp từ nhiều nguồn thông tin phân tán. Điều này yêu cầu hệ thống phải hiểu được mối quan hệ phức tạp giữa các thực thể và cách chúng tương tác với nhau.

Dữ Liệu:

- Tập dữ liệu sẽ sử dụng được lấy từ Repo Github sau: [multi-hop RAG task](#).
- Thông tin chung về tập dữ liệu:
 - Là bộ dữ liệu multi-hop QA gồm 25K queries mà thông tin để trả lời câu hỏi rải rác từ 2-4 documents được tạo ra bằng cách ghép nhiều câu hỏi single-hop từ các bộ dữ liệu khác.
- Gồm ba file dữ liệu json ở folder musique__dataset là :
 - `./musique__dataset/queries.json`: Gồm tập câu queries
 - `./musique__dataset/corpus.json`: Gồm tập các documents
 - `./musique__dataset/dev_rel_docs.json`: Dữ liệu nối giữa queries và groundtruth documents cho queries đó.

2 Các bước thực hiện

2.1 Pip install và import những thư viện cần thiết

```
[ ]: # !pip install nest_asyncio
# !pip install lightrag-hku
# !pip install asyncio
# !pip install together
# !pip install -U FlagEmbedding
# !pip install pyvis
```

```
[3]: import os
import json
import sys
sys.path.append("../..")
import nest_asyncio
nest_asyncio.apply()
import os
import asyncio
from lightrag import LightRAG, QueryParam
from lightrag.kg.shared_storage import initialize_pipeline_status
from lightrag.utils import setup_logger
```

Tập dữ liệu nằm trong folder musique_dataset Folder gồm ba file là:

- queries.json
- corpus.json
- dev_rel_docs.json

```
[4]: musique_corpus_file = "./musique_dataset/corpus.json"
musique_queries_file = "./musique_dataset/queries.json"
musique_query_docs_file = "./musique_dataset/dev_rel_docs.json"
```

2.2 Tiền xử lý dữ liệu

1. Load dữ liệu từ file json

```
[5]: # Đọc dữ liệu hai file:
## Load dữ liệu multi-hop queries từ file multi_hop_rag_file
with open(musique_corpus_file) as f:
    musique_corpus = json.load(f)

## Load corpus từ file multi_hop_corpus_file
with open(musique_queries_file) as f:
    musique_queries = json.load(f)

## Load corpus từ file multi_hop_corpus_file
with open(musique_query_docs_file) as f:
```

```
musique_query_docs = json.load(f)
```

2. Lấy 10 queries đầu tiên và lấy những groundtruth documents cho các query đó

```
[6]: number_of_queries = 10
sub_musique_queries = {k: v for k, v in
                        list(musique_queries.items())[:number_of_queries]}
total_queries = list(sub_musique_queries.values())

# Tạo sub-corpus bao gồm những văn bản cần để trả lời 10 queries đầu tiên:
all_relevant_doc_ids = set()
for query_id in sub_musique_queries:
    all_relevant_doc_ids.update(musique_query_docs[query_id])

total_corpus = [musique_corpus[doc_id] for doc_id in all_relevant_doc_ids]

# In tổng số lượng văn bản và số lượng queries truy vấn trong corpus
print(f"Tổng số lượng queries: {len(sub_musique_queries)}")
print(f"Tổng số lượng văn bản: {len(total_corpus)}")
```

Tổng số lượng queries: 10

Tổng số lượng văn bản: 28

2.3 Indexing Graph

- Trong thư viện lighrag có 4 thành phần phải được xác định:
 - **Mô hình lớn sử dụng.** Mặc định thư viện là gpt-4o từ OpenAI
 - **Mô hình embedding.** Mặc định thư viện là text-embedding-3-small từ OpenAI
 - **Vector database.** Mặc định thư viện là dùng NanoVectorDB
 - **Graph database.** Mặc định thư viện là dùng base trên thư viện networkx

1 Custome LLM

- Gọi LLM từ Together AI thông qua gọi API
- **Together AI**
 - Là nền tảng cung cấp truy cập đến nhiều mô hình LLM mã nguồn mở
 - Đăng ký tài khoản tại: [TogetherAI](#)
 - Mỗi tài khoản mới được free 1\$ credits

```
[7]: # Gọi Api mô hình từ Together.ai
together_api = "<YOUR_TOGETHER_API_KEY>"
os.environ["TOGETHER_API_KEY"] = together_api
os.environ["LLM_MODEL_NAME"] = "meta-llama/Llama-3.3-70B-Instruct-Turbo"
```

```
[10]: from together import AsyncTogether
from lightrag.base import BaseKVStorage
from lightrag.utils import compute_args_hash

async def together_complete_if_cache(
    model, prompt, system_prompt=None, history_messages=[], **kwargs
) -> str:
    together_client = AsyncTogether(api_key = os.getenv("TOGETHER_API_KEY"))
    hashing_kv: BaseKVStorage = kwargs.pop("hashing_kv", None)
    messages = []
    if system_prompt:
        messages.append({"role": "system", "content": system_prompt})
    messages.extend(history_messages)
    messages.append({"role": "user", "content": prompt})
    if hashing_kv is not None:
        args_hash = compute_args_hash(model, messages)
        if_cache_return = await hashing_kv.get_by_id(args_hash)
        if if_cache_return is not None:
            return if_cache_return["return"]

    response = await together_client.chat.completions.create(
        model=model, messages=messages, **kwargs
    )
    if hashing_kv is not None:
        await hashing_kv.upsert(
            {args_hash: {"return": response.choices[0].message.content,
                        "model": model}}
        )
        await hashing_kv.index_done_callback()
    return response.choices[0].message.content

async def together_llm_complete(
    prompt, system_prompt=None, history_messages=[], **kwargs
) -> str:
    return await together_complete_if_cache(
        os.getenv("LLM_MODEL_NAME"),
        prompt,
        system_prompt=system_prompt,
        history_messages=history_messages,
        **kwargs,
    )
```

2 Custome Embedding

- Sử dụng mô hình BGE-M3 từ thư viện : [FlagEmbedding](#).

```
[ ]: from FlagEmbedding import BGEM3FlagModel
from lightrag.utils import wrap_embedding_func_with_attrs
```

```

import numpy as np

# Sử dụng embedding model là bge-m3
EMBED_MODEL = BGEM3FlagModel("BAAI/bge-m3",
                              cache_folder="bge-m3",
                              use_fp16=True,
                              pooling_method='mean',
                              devices="cpu")

#BGE-M3 có số chiều là 1024 và set up max tokens là 8192 tokens
embedding_dimension = 1024
max_tokens = 8192

@wrap_embedding_func_with_attrs(
    embedding_dim=embedding_dimension,
    max_token_size=max_tokens,
)
async def bge_m3_embedding(texts: list[str]) -> np.ndarray:
    embeddings = EMBED_MODEL.encode(texts,
                                     return_dense=True)['dense_vecs']

    return embeddings

```

3 Custom prompts:

- Dịch và sử dụng prompts bản tiếng việt
- Đảm bảo cho việc extract thực thể nó chính xác hơn cho văn bản tiếng việt

```

[14]: # Use the vietnamese prompts:
from prompt.prompt_vi import PROMPTS as PROMPTS_VI
from lightrag.prompt import PROMPTS
for name_prompt, value_prompt in PROMPTS.items():
    name_prompt_vi = name_prompt + "_VI"
    PROMPTS[name_prompt] = PROMPTS_VI[name_prompt_vi]

```

4 Xây dựng đồ thị từ văn bản

```

[ ]: setup_logger("lightrag", level="INFO")
WORKING_DIR = "./rag_storage"
if not os.path.exists(WORKING_DIR):
    os.mkdir(WORKING_DIR)

##### TODO: Thực hành #####
# Yêu cầu:
# + Input:
#   - Khởi tạo LightRAG object
#   - Thay thế LLM bằng hàm together_llm_complete() gọi Together AI
#   - Thay thế embedding model bằng BGE-M3 model
#   - Thực hiện quá trình insert các chunks (total_corpus) và xây dựng graph

```

```

# - Quá trình có thể kéo dài 3-5 p
# + Output:
# - Xây dựng graph thành công và trả về object lightrag
# để phục vụ truy vấn trên đồ thị sau.
# Tham khảo cách sử dụng tại: https://github.com/HKUDS/LightRAG

##### End TODO #####
async def initialize_rag():

    #####
    ### START CODE HERE ###
    #####

    return rag

async def insert():

    #####
    ### START CODE HERE ###
    #####

    return rag

rag = asyncio.run(insert())

```

- Sau khi xây xong mọi dữ liệu về đồ thị, chunks, vector database được lưu trong **WORKING_DIR**
- **WORKING_DIR** trong bài này được set bằng `./rag_storag`
- Trong folder **WORKING_DIR** gồm:
 - file **graph_chunk_entity_relation.graphml** chứa thông tin về graph.
 - file **kv_store_full_docs.json** chứa thông tin về documents gốc
 - file **kv_store_text_chunks.json** chứa thông tin về các chunks được cắt từ documents
 - file **vdb_chunks.json** lưu embedding của các chunks
 - file **vdb_entities.json** lưu embedding của entities (thực thể).
 - file **vdb_relationships.json** lưu embedding của relationships (mối quan hệ).

2.4 Information of knowledge Graph and Visualizing Graph

1 Trích xuất một số thông tin về Graphs

- Số node và edges trong knowledge Graph
- Lấy ra một node và một cạnh bất kì trong graph và in ra những attributes trong nút/cạnh đó

```

[ ]: import networkx as nx

def print_graph_info(graphml_file: str) -> None:

```

```

##### TODO: Thực hành #####
# Yêu cầu:
# + Input:
#   - Dùng thư viện networkx để load đồ thị từ file .graphml trong
#     folder WORKING_DIR của graphrag
#   - In ra số lượng nút, số lượng cạnh trong đồ thị
#   - In ra thông tin của nút và cạnh bất kì trong đồ thị
# + Output:
#   - Số lượng nút
#   - Số lượng cạnh
#
#   - Thông tin một nút bất kì gồm: tên nút,
#     và một số đặc tính của nút như entity type, description
#     document_id gốc chứa nút thực thể đó
#
#   - Thông tin một cạnh bất kì gồm:
#     + Tên nút đầu của cạnh
#     + Tên nút cuối của cạnh
#     + Trọng số của cạnh
#     + Description chứa cạnh đó
#     + document_id gốc chứa nút cạnh đó
# Tham khảo cách sử dụng tại: https://networkx.org/documentation/stable/
↪tutorial.html

#####
### START CODE HERE ###
#####

##### End TODO #####

return None

graph_path = "./rag_storage/graph_chunk_entity_relation.graphml"
print_graph_info(graph_path)

```

2 Visualize Knowledge Graph đã tạo

- **Pyvis** là một thư viện python để trực quan hóa đồ thị.
- Nó có thể trực quan hóa đồ thị từ thư viện python quản lý đồ thị như Networkx,...

```

[ ]: import networkx as nx
from pyvis.network import Network
import webbrowser
import ast

# Load file GraphML từ save folder
graphml_file = "./rag_storage/graph_chunk_entity_relation.graphml"

```

```

G = nx.read_graphml(graphml_file)

# Khởi tạo Pyvis network
net = Network(
    height="100vh",
    width="100vw",
    bgcolor="black",
    font_color="white",
    notebook=True
)

# Add nodes từ graph của networkx sang Pyvis
for node in G.nodes(data=True):
    node_id, attrs = node
    title = "\n".join([f"{key}: {value}" for key, value in attrs.items()])
    net.add_node(
        node_id,
        label=node_id,
        title=title,
        size=20
    )

for edge in G.edges(data=True):
    source, target, attrs = edge
    title = "\n".join([f"{key}: {value}" for key, value in attrs.items()])
    net.add_edge(
        source,
        target,
        title=title,
        color="#FFFFFF",
        width=2
    )

# Save and show graph
net.show("graph.html")
webbrowser.open('graph.html')

```

2.5 Truy vấn NaiveRAG và truy vấn trên đồ thị

1 Lấy ra tập câu sub-queries ở phía trên với groundtruth documents cho mỗi queries

```

[ ]: query_lists = sub_musique_queries.values()
query_to_gt_docs = []
for query_id, query_content in sub_musique_queries.items():
    groundtruth_doc_ids = musique_query_docs[query_id]
    grountruth_contents = [musique_corpus[doc_id] for
                           doc_id in groundtruth_doc_ids]

```



```
query_to_gt_docs.append((query_content, grountruth_contents))
```

2 Lấy ra một câu test query bất kì để thực hiện truy vấn

```
[ ]: test_query, test_gt_documents = query_to_gt_docs[1]
print("Câu queries: ")
print(test_query)
print("Văn bản chứa câu trả lời: ")
for i, doc in enumerate(test_gt_documents):
    print(f"Văn bản: {i}")
    print(doc)
```

3 Tìm những chunks liên quan tới query

```
[ ]: import asyncio
def search_relevant_chunks(query: str,
                           graphrag : LightRAG,
                           top_k = 5) -> list[str]:
    # Lấy ra vector database lưu chunks
    chunk_vdb = graphrag.chunks_vdb

    # Từ chunk_vdb lấy ra top_k chunk_id liên quan đến query nhất
    result = asyncio.run(chunk_vdb.query(query, top_k = top_k))
    result_chunk_id = [sample["__id__"] for sample in result]
    # Load database lưu thông tin về chunk:
    chunk_kv = graphrag.text_chunks

    chunk_objects = asyncio.run(chunk_kv.get_by_ids(result_chunk_id))

    # Trích xuất nội dung của chunk từ chunk object

    chunk_contents = [sample["content"] for sample in chunk_objects]

    return chunk_contents
```

```
[ ]: search_relevant_chunks(test_query, rag, top_k = 10)
```

4 Tìm những thực thể (entities) liên quan tới query

```
[ ]: import asyncio
def search_relevant_entities(query: str,
                             graphrag : LightRAG,
                             graph_path : str,
                             top_k = 5) -> list[dict[str, str]]:
    # Lấy ra vector database lưu entities
    entities_vdb = graphrag.entities_vdb
    # Từ entities_vdb lấy ra top_k kết quả liên quan đến query nhất
    # từ entities vdb
```

```

result = asyncio.run(entities_vdb.query(query, top_k = top_k))
G = nx.read_graphml(graph_path)
all_entities_information = []

##### TODO: Thực hành #####
# Yêu cầu:
# + Input:
#   - Tìm ra tất cả tên những thực thể liên quan đến query
#   - Từ tên các thực thể lấy tất cả những thông tin liên quan
#     đến thực thể đó bao gồm:
#       + Tên thực thể
#       + Loại thực thể đó
#       + Tất cả Miêu tả thực thể
#       + Bậc của nút thực thể đó
# + Output:
#   - Format là một list của các dictionaries:
#       + key là tên đặc điểm
#       + value là giá trị của đặc điểm đó.
#   - Ví dụ:
#       [
#         {"entity_name" : "...",
#          "entity_type" : "...",
#          "description_1" : "...",
#          "description_2" : "...",
#          "node_degree" : "..."},
#         ...
#       ]
# Gợi ý: Tham khảo cách sử dụng tại: https://github.com/gusye1234/
# https://networkx.org/documentation/
# stable/tutorial.html

#####
### START CODE HERE ###
#####

##### End TODO #####

return all_entities_information

```

```

[ ]: graph_path = "./rag_storage/graph_chunk_entity_relation.graphml"
search_relevant_entities(test_query,
                        rag,
                        graph_path,
                        top_k = 5)

```

5 Tìm những mối quan hệ (relationships) liên quan tới query

```
[ ]: import asyncio
def search_relevant_relations(query: str,
                             graphrag : LightRAG,
                             graph_path : str,
                             top_k = 5) -> list[dict[str, str]]:

    # Lấy ra vector database lưu relationships
    relationships_vdb = graphrag.relationships_vdb
    # Từ relationships_vdb lấy ra top_k tên thực thể liên quan đến query nhất
    result = asyncio.run(relationships_vdb.query(query, top_k = top_k))
    all_edges_information = []

    ##### TODO: Thực hành #####
    # Yêu cầu:
    # + Input:
    #   - Tương tự bài tập trên tìm tất cả thông tin
    #     cạnh liên quan nhất đến queries
    # + Output:
    #   - Format là một list của các dictionaries:
    #       + key là tên đặc điểm
    #       + value là giá trị của đặc điểm đó.
    #   - Ví dụ:
    #       [
    #           {"Node source" : "...",
    #            "Node head" : "...",
    #            "description_1" : "...",
    #            "description_2" : "...",
    #            "Edge weight" : "..."},
    #           ...
    #       ]
    # Tham khảo cách sử dụng tại: https://github.com/gusye1234/nano-vectordb
    #                               https://networkx.org/documentation/stable/
    ↪ tutorial.html

    #####
    ### START CODE HERE ###
    #####

    ##### End TODO #####

    return all_edges_information

[ ]: graph_path = "./rag_storage/graph_chunk_entity_relation.graphml"
search_relevant_relations(test_query,
                          rag,
                          graph_path, top_k = 5)
```

6 Trả lời câu query bằng 2 phương pháp là :

- Naive RAG thông thường
- Phương pháp hybrid(local + global) search trong LightRAG.

```
[ ]: # Naive RAG query
##### TODO: Thực hành #####
# Yêu cầu:
# + Input:
#   - Thực hiện truy vấn và trả ra câu trả lời cho câu test_query.
#   - Thực hiện truy vấn bằng hai cách là
#       + naive search (only text chunks)
#       + mix search   (kết hợp giữa local search và
#                       global search trên đồ thị)
# + Output:
#   Lưu ra câu trả lời của hai phương pháp ra hai biến là:
#       - naive_answer
#       - hybrid_answer
# Gợi ý: Tham khảo cách sử dụng tại: https://github.com/HKUDS/LightRAG

# Naive search

#####
### START CODE HERE ###
#####

naive_answer = None
print(naive_answer)

# Mix search

#####
### START CODE HERE ###
#####

hybrid_answer = None
print(hybrid_answer)

##### End TODO #####
```

2.6 Đánh giá câu trả lời của Naive RAG và GraphRAG bằng LLM

- Trong phần này, ta sẽ dùng LLM để đánh giá hai câu trả lời của NaiveRAG và LightRAG Hybrid dựa trên các tiêu chí sau:
 - **Tính toàn diện (Comprehensiveness)** : Đánh giá xem câu trả lời có thể trả lời câu hỏi 1 cách toàn diện hay không
 - **Tính đa dạng (Diversity)** : Đánh giá xem câu trả lời có đa dạng và đưa nhiều thông

tin đa chiều về câu hỏi không

- **Empowerment** : Câu trả lời giúp người đọc hiểu và đưa ra đánh giá sáng suốt về chủ đề như thế nào ?

```
[ ]: from prompt.evaluation_prompt import EVALUATION_PROMPT
# In ra prompt để đánh giá giữa 2 câu trả lời
print(EVALUATION_PROMPT)
```

```
[ ]: from together import Together
def evaluate_answer(answer_1: str,
                    answer_2: str,
                    query: str,
                    gt_docs: list[str]) -> str:

    ##### TODO: Thực hành #####
    # Yêu cầu:
    # + Input:
    #   - Từ câu trả lời 1, câu trả lời 2, query
    #     và documents chứa đoạn thông tin chính xác
    #
    #   - Đánh giá câu trả lời nào tốt hơn dựa trên 3 tiêu chí

    #   - Sử dụng EVALUATION_PROMPT ở phía trên kèm theo 2 câu trả lời
    #     sinh ra bởi naive và mix search, đánh giá 2 câu trả lời đó

    #   - Sử dụng mô hình LLM từ Together AI để đánh giá

    #   - Gợi ý: Nên dùng mô hình LLM to ví dụ meta-llama/Llama-3.
    ↪ 3-70B-Instruct-Turbo

    # + Output:
    #   - Text đánh giá nhận xét về 2 câu trả lời do LLM sinh ra
    # Gợi ý: Tham khảo cách sử dụng tại: https://docs.together.ai/docs/
    ↪ quickstart

    ##### START CODE HERE #####

    ##### End TODO #####

    pass
```

```
[55]: print(evaluate_answer(naive_answer, hybrid_answer, test_query,
    ↪ test_gt_documents))
```

```
{
  "Comprehensiveness": {
    "Winner": "Answer 2",
```

"Explanation": "Answer 2 cung cấp nhiều thông tin chi tiết hơn về các cuộc thảo luận Tam bên, bao gồm cả thời gian bắt đầu (giữa tháng 6) và các quốc gia tham gia (Anh, Liên Xô và các quốc gia khác). Ngoài ra, Answer 2 cũng cung cấp thông tin về các bảo đảm tiềm năng cho các nước Trung và Đông Âu, cũng như quan điểm của Liên Xô và Anh trong các cuộc đàm phán. Answer 1 chỉ cung cấp thông tin cơ bản về thời gian bắt đầu và quốc gia tham gia, nhưng không đi sâu vào các chi tiết cụ thể của các cuộc thảo luận."

},

"Diversity": {

"Winner": "Answer 2",

"Explanation": "Answer 2 cung cấp nhiều quan điểm và thông tin đa dạng hơn về các cuộc thảo luận Tam bên. Answer 2 đề cập đến các bảo đảm tiềm năng cho các nước Trung và Đông Âu, quan điểm của Liên Xô và Anh, cũng như thông tin về Tổ chức Hiệp ước Warsaw và Szlachta. Điều này cho thấy Answer 2 có sự đa dạng và phong phú hơn trong việc cung cấp thông tin và quan điểm khác nhau. Answer 1 chỉ tập trung vào việc xác định quốc gia và thời gian bắt đầu của các cuộc thảo luận, mà không cung cấp nhiều thông tin đa dạng."

},

"Empowerment": {

"Winner": "Answer 2",

"Explanation": "Answer 2 giúp người đọc hiểu rõ hơn về các cuộc thảo luận Tam bên và các vấn đề liên quan. Answer 2 cung cấp thông tin chi tiết và đa dạng, giúp người đọc có thể hình dung rõ hơn về các sự kiện và quan điểm khác nhau. Điều này cho phép người đọc có thể đưa ra các quyết định và đánh giá thông tin một cách thông minh hơn. Answer 1 chỉ cung cấp thông tin cơ bản, mà không giúp người đọc hiểu rõ hơn về các vấn đề liên quan."

},

"Overall Winner": {

"Winner": "Answer 2",

"Explanation": "Answer 2 là overall winner vì nó cung cấp thông tin chi tiết và đa dạng hơn về các cuộc thảo luận Tam bên, giúp người đọc hiểu rõ hơn về các vấn đề liên quan. Answer 2 cũng cung cấp nhiều quan điểm và thông tin đa dạng hơn, giúp người đọc có thể đưa ra các quyết định và đánh giá thông tin một cách thông minh hơn. Tổng thể, Answer 2 đáp ứng tốt hơn các tiêu chí về Comprehensiveness, Diversity và Empowerment so với Answer 1."

}

}